TA: Achyuth Gundrapally
TA Hours: Wednesday, 2:00-4:00 PM

Lab 4: Design Techniques to Accelerate the Convolutional Layer of DNNs on FPGA

Case study : First Convolutional Layer in VGG-11

Assigned: Tuesday, October 20, 2026
Due date: Thursday, October 29, 2026, 7:00 PM

Contents

1 Case study : First Convolutional Layer in VGG-11 **1**

2 Design Techniques **3**
2.1 Convolutional Tiling . . . 3
2.2 Numeric Precision : Float vs Fixed Point . . . 4

3 Your Tasks **6**
3.1 Part 1: Un-Optimized Tiled Convolution 25 points . . . 6
3.1.1 Code Organization . . . 6
3.1.2 Your Task . . . 6
3.2 Part 2: Exploring 20 points . . . 7
3.2.1 Code Organization . . . 7
3.2.2 Your Task . . . 7
3.3 Part 3: Optimized Tiled Convolution 15 points . . . 8
3.3.1 Code Organization . . . 8
3.3.2 Your Task . . . 8
3.4 Part 4: Fixed Point Exploration 20 points . . . 8
3.4.1 Code Organization . . . 8
3.4.2 Your Task . . . 9
3.5 Part 5: Build the HLS IP and deploy on FPGA 20 points . . . 9
3.5.1 Code Organization . . . 9
3.5.2 Your Task . . . 9

4 What to Submit **9**

1 Case study : First Convolutional Layer in VGG-11

The VGG model is a type of deep convolutional neural network (CNN) architecture, developed by the Visual Geometry Group (VGG) at the University of Oxford. It became widely known after performing well in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2014. The most popular variants are VGG-16 and VGG-19, where the numbers indicate the total number of layers. Figure .1 is an example of VGG-16 Architecture.

Key Features of the VGG Model:

- Architecture: VGG consists of sequential convolutional layers with small receptive fields (3x3 filters) stacked on top of each other, followed by max-pooling layers and fully connected layers at the end.
- Fixed Filter Size: All convolutional layers use 3x3 filters with a stride and padding of 1 to preserve spatial resolution.
- Max Pooling: After every two or three convolution layers, a 2x2 max-pooling layer with a stride of 2 is applied to reduce the spatial dimensions.
- Fully Connected Layers: At the end, there are typically two fully connected layers, followed by a softmax layer for classification.

VGG-11 Architecture Example

- Input: 3×224×224 image.
- Several layers of 3x3 convolution and 2x2 max-pooling.
- 11 weight layers (8 convolutional + 3 fully connected).

Pros of the VGG Model

- Simplicity: The architecture is simple, based on stacking small-sized filters, making it easy to understand and implement on hardware, and in our case coding in high-level synthesis.
- Deep Layers: With 16-19 layers, VGG is deeper than previous networks like AlexNet, allowing it to capture more complex features. Higher accuracy!

Cons of the VGG Model

- Memory Intensive: VGG has a large number of parameters (132863336 for VGG-11), making it memory-intensive. Which leads to higher memory requirements and slower inference times.
- Computational Cost: VGG's depth and the fully connected layers at the end make it computationally expensive, which again will require more time for both training and inference.

Due to its high computational cost and memory usage, VGG is not ideal for resource-constrained hardware platforms such as FPGAs which makes this model a very interesting case to accelerate and deploy on FPGAs for us! Yay!.

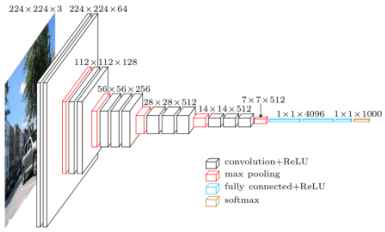


Figure 1: VGG-16 Architecture

2 Design Techniques

2.1 Convolutional Tiling

One of the biggest challenges that comes with accelerating neural network models on FPGAs is the **scalability** of the model, (e.g., large input size, and/or large number of the one layer parameters). A very well known technique to solve this is to split the input image into **tiles** and/or split the number of kernels into **groups** and run the convolution operation between the one individual tile and the groups of kernels one after the other. The overall latency is then the latency of the convolution operation between the [(one individual tile × groups of kernels) × the number of tiles in the input image].

Figure .2 shows the regular convolutional operation between the input feature map and the kernels, notice that the buffer sizes of the input, weights, and output in the programmable logic are basically the size of the original input fmap and the required parameters and the expected output feature map size respectively.

However, Figure .3 shows the tiled convolutional operation between the first tile in the input feature map and the first group of kernels, notice that the buffer sizes of the input, weights, and output in the programmable logic are basically the size of the (one tile + MARGIN), the required group parameters and the expected output fmap size respectively.

Hints to implement the Tiled Convolution

- THE MARGIN HERE IS THE EXTRA SPACE WE NEED AFTER PADDING THE INPUT FEATURE MAP
- YOU MUST PAY ATTENTION TO THE LOCATION OF THE TILE IN THE INPUT FEATURE MAP, YOU DON'T WANT TO END UP PADDING THE TILES THAT SHOULDN'T BE PADDED ! THINK ABOUT THIS!

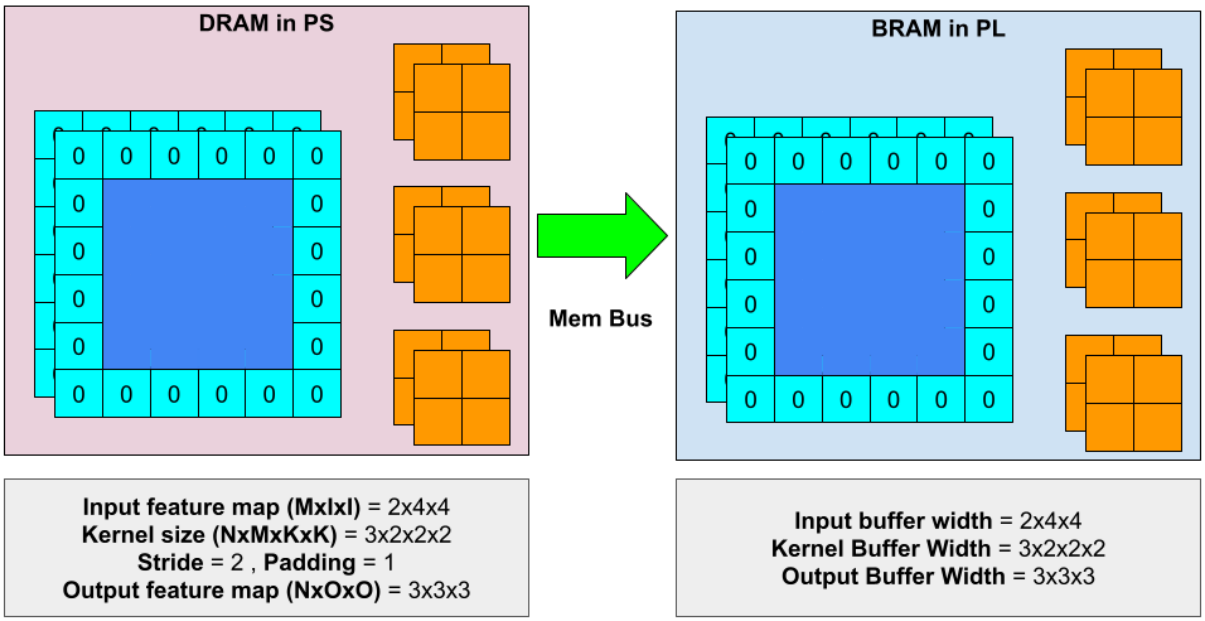


Figure 2: Example of Regular Convolution

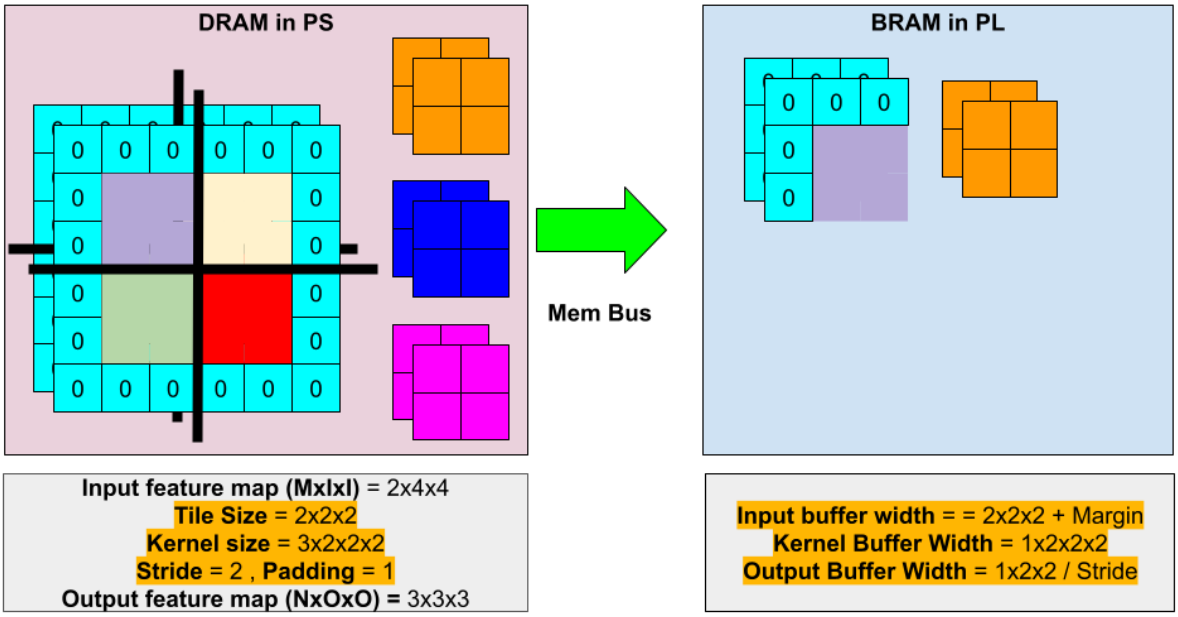


Figure 3: Example of Tiled Convolution

2.2 Numeric Precision : Float vs Fixed Point

Fixed-point arithmetic is a key concept in High-Level Synthesis, and a designer should consider this technique when designing hardware accelerators on FPGAs. Using fixed-point arithmetic instead of floating-point can significantly improve the performance, reduce resource usage, and minimize power consumption on FPGA devices.

What is Fixed-point Arithmetic? In fixed-point arithmetic, numbers are represented with a fixed number of digits before and after the decimal point. Unlike floating-point arithmetic, which uses scientific notation to represent numbers (e.g., IEEE single/double precision). Fixed-point representations allocate a fixed number of bits for the integer part and a fixed number for the fractional part. This is useful in hardware design because it allows for faster and more resource-efficient operations on FPGAs.

Pros of Using Fixed-point in Vitis HLS:

- Efficient Resource Usage: Fixed-point arithmetic consumes fewer FPGA resources compared to floating-point.
- Lower Latency: Fixed-point operations are faster than floating-point operations, resulting in reduced latency, which is critical in AI applications applications.
- Low Power: less computational complexity leads to lower power consumption.
- Customizable Precision: A designer can control the number of bits allocated for both the integer and fractional parts, providing a fine-grained trade-off between precision and resource usage.

Cons of Using Fixed-point in Vitis HLS:

- Manual Tuning Required: Selecting the appropriate number of bits for the integer and fractional parts can be tricky and might require careful analysis and experimentation to avoid overflow or underflow.
- Limited Range: Fixed-point numbers have a smaller dynamic range compared to floating-point, making them less suitable for applications that require representing very large or very small numbers.
- Quantization Errors: Converting from floating-point to fixed-point introduces quantization errors, which can affect the accuracy of the results.

Figures .4 and 5 shows examples how to represent the integers and fixed point numbers in VitisHLS.

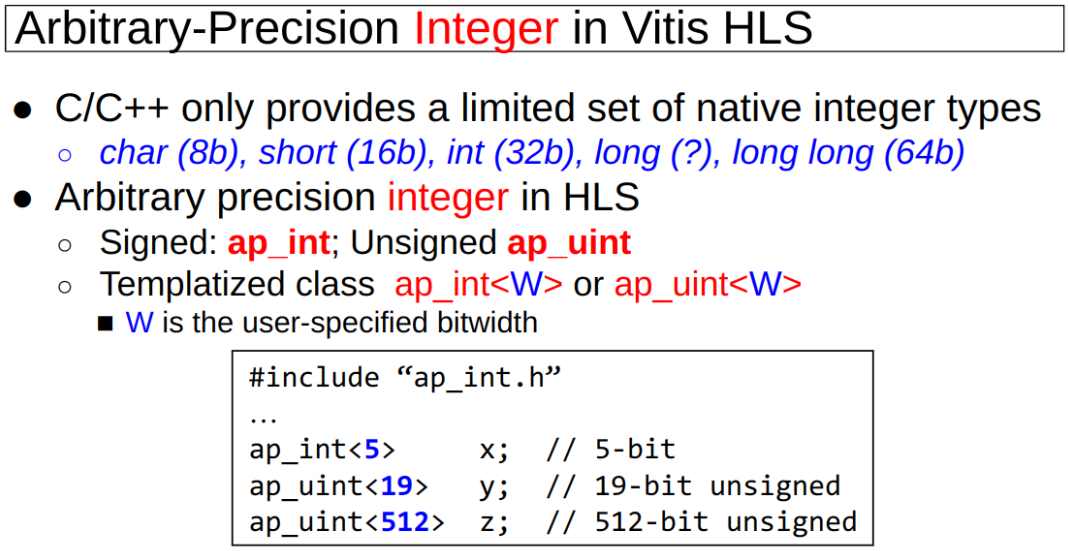


Figure 4: Arbitrary-Precision Integer in VitisHLS

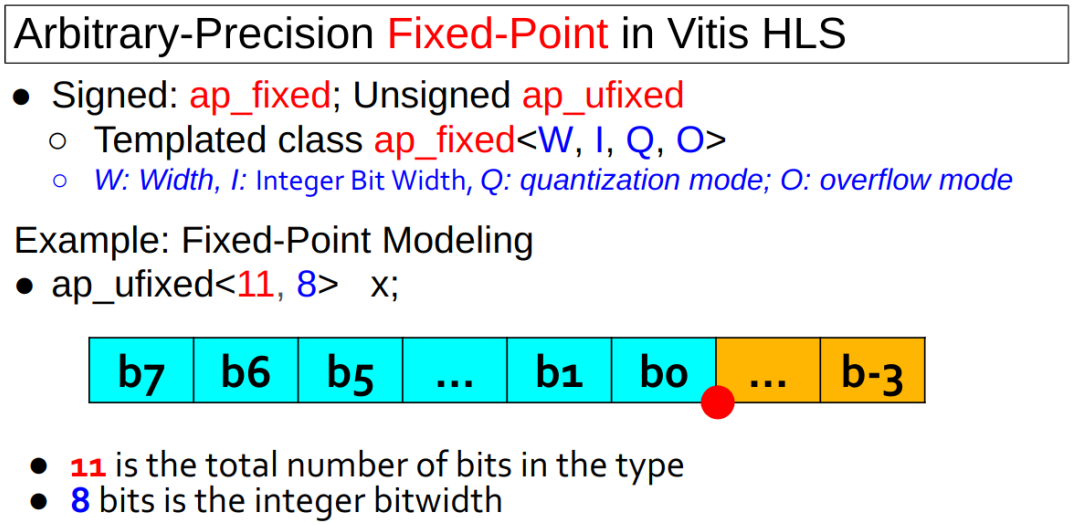


Figure 5: Arbitrary-Precision Fixed-Point in VitisHLS

3 Your Tasks

3.1 Part 1: Un-Optimized Tiled Convolution 25 points

3.1.1 Code Organization

You are given a zip file named lab4.zip on the Canvas site, navigate to part1 folder which contains the following files for you to build the part 1 project.

- **part1/utils.hpp:** the header file of the functions needed to help in the hardware implementation of the tiled convolution
- **part1/utils.cpp:** the source file of the functions needed to help in the hardware implementation of the tiled convolution
- **part1/tiled_conv.hpp:** the header file of the tiled convolution implementation
- **part1/tiled_conv.cpp:** the source file of the tiled convolution implementation
- **part1/tb_tiled_conv.cpp:** a test bench file that helps verify the code.
- **part1/vitis_hls.tcl:** a script file to automatically simulate, synthesize and build the the HLS IP, and specify the FPGA device and the clock cycle we are targeting. **We are targeting the Pynq-Z2 FPGA board with a clock cyle of 10ns.**
- **part1/Makefile:** a Makefile for you to easily :
  (1) simulate your program : just type **make csim**
  (2) synthesize your design : just type **make csyn**
  (3) build the HLS ip : just type **make csyn** (YOU MUST UNCOMMENT THE *export command* in the vitis_hls.tcl file to build the IP)
  (4) clean the project : just type **make clean**

3.1.2 Your Task

In this part of the lab, you have to implement an unoptimized but synthesizable code that performs tiling-based convolution of the first convolution layer of VGG-11.

1. This part of the lab is simulation only. You do not have to run Vitis synthesis. You do not need to add optimization pragmas in this part of the lab. The goal is just to implement a correctly tiling-based convolution layer implementation that is functionally correct and establish a baseline.

2. For the baseline, we are going to split the input image into 1024 tiles of size 7×7 each, and the 64 kernels into 16 groups of 4 kernels each. THIS IS THE BASELINE, DO NOT CHANGE THOSE VALUES NOW. There is 64×224×224 comparison point, and again we are going to use the Mean-squared error, the MSE should be less than 5. YES I TESTED THIS A LOT OF TIMES, FOR ALL THE DIFFERENT POSSIBLE TILING HARDWARE VALUES, MSE SHOULD BE LESS THAN 5.

3. There are 5 functions you need to implement in 2 files, in addition of assigning values to the tiled convolution hardware in (tiled_conv.hpp) file, the functions needs to be implemented are below:

(a) load_input_tile_from_DRAM (in utils.cpp) 5 points
(b) load_layer_params_from_DRAM (in utils.cpp) 5 points
(c) conv_3x3 (in utils.cpp) 5 points
(d) store_output_tile_to_DRAM (in utils.cpp) 5 points
(e) tiled_conv (in tiled_conv.cpp) 5 points

3.2 Part 2: Exploring 20 points

3.2.1 Code Organization

In this part of the lab, you are going to work on the same code from part1.

3.2.2 Your Task

After implementing a correct tiled convolution function, now it is time to explore how different sizes of the tiled blocks and/or numbers of kernel groups convolution affect the resource usage on FPGA. The latency and resource-utilization numbers of the baseline you implemented in part1 is provided in Table .1.

Latency	BRAM	DSP	FF	LUT
18220034	54	71	18310	21974

Table 1: initial latency and resource utilization on the baseline implementation

Very quick let us just calculate the number of possible experiments here:

1. Number of Kernels 64: factors of 64; (1, 2, 4, 8, 16, 32, 64)
   Note that the Kernel groups must be divided equally

2. Input image size (3×224×224) : factors of 224; (4, 7, 8, 14, 16, 28, 32, 56, 112)
   Note that the Tile size must be at least equal to the kernel size

So the number of possible implementations of this tiled convolution function is 63. OMG! Are we really going to synthesize this 63 times? This is really up to you. I would start first by synthesizing the baseline, and report my numbers and compare it with Table .1. After that start by increasing the tile size only, and keep the number of kernel groups constant for now and see how the latency and resource usage changes, is it better? Worse? do you need to size the tile bigger/smaller? Draw a conclusion here. After that start by increasing the number of kernel groups, and keep the size of the Tile blocks constant, and see how the latency and resource usage changes, is it better? Worse? do you need to increase/decrease the size of kernel groups? Draw a conclusion here. You might need to explore and synthesize this around 15 times to draw a very solid conclusion and find/choose your golden hardware implementation of the Tiled Convolution, that will be used to optimize in part 3.

Use tables, graphs, pie charts, etc . . . whatever you find necessary to report your findings, the goal is to easily visualize your findings. Draw a very solid conclusion, and comment on how the relationship between the tile size and/or the number of kernel groups affect the latency and resource utilization. I CAN'T STRESS THIS ENOUGH, I NEED TO READ WHY YOU ARE DOING WHAT YOU DECIDED TO DO. DON'T JUST ATTACH FIGURES WITHOUT EXPLANATIONS.

3.3 Part 3: Optimized Tiled Convolution 15 points

3.3.1 Code Organization

In this part of the lab, you are going to work on the same code from part1.

3.3.2 Your Task

After exploring in part2 how different sizes of the tiled blocks and/or numbers of kernel groups convolution affect the latency and resource usage on FPGA, and choosing your golden hardware implementation of the Tiled Convolution. Now it is the time to optimize the code with your favorite pragmas. You need to optimize the design by inserting (pragmas) to minimize the latency ensuring the resource utilization are all under 100%. Focus on exploring the effect of the three main pragmas discussed in lab2 (pipeline, unroll, array partition) in your design and apply them appropriately to minimize the latency of the synthesized design.

3.4 Part 4: Fixed Point Exploration 20 points

3.4.1 Code Organization

From the zip file named lab4.zip on the Canvas site, navigate to part4 folder which contains the following files for you to build the part 4 project.

- **part4/utils.hpp:** the header file of the functions needed to help in the hardware implementation of the fixed point tiled convolution
- **part4/utils.cpp:** the source file of the functions needed to help in the hardware implementation of the fixed point tiled convolution
- **part4/tiled_conv.hpp:** the header file of the fixed point tiled convolution
- **part4/tiled_conv.cpp:** the source file of the fixed point tiled convolution
- **part4/tb_tiled_conv.cpp:** a test bench file that helps verify the code.
- **part4/vitis_hls.tcl:** a script file to automatically simulate, synthesize and build the the HLS IP, and specify the FPGA device and the clock cycle we are targeting. **We are targeting the Pynq-Z2 FPGA board with a clock cyle of 10ns.**
- **part4/Makefile:** a Makefile for you to easily :
  (1) simulate your program : just type **make csim**
  (2) synthesize your design : just type **make csyn**
  (3) build the HLS ip : just type **make csyn** (YOU MUST UNCOMMENT THE *export command* in the vitis_hls.tcl file to build the IP)
  (4) clean the project : just type **make clean**

For the most part, it is almost the same code in part 1, however here we are using a fixed point data type instead of floating points to represent the feature maps and the parameters (weights and biases). The baseline provided is 16 bit fixed point with 4 fractional bits. Copy and use your code from part1. **Note : depending on how you implemented the 4 functions in the utils.cpp file, you might need to change the datatype for some of those implementations, so you run the simulation successfully.**

3.4.2 Your Task

In this part of the lab, you will take your optimized golden tiled convolution layer you worked on in part3, and simulate, synthesis and explore how different fixed point datatype of the feature maps and parameters affect the latency and resource utilization. Table .2 shows the latency and resource usage of the baseline fixed point datatype. Take this as a baseline, and start building your experiments and findings.

Latency	BRAM	DSP	FF	LUT
17328130	40	29	9725	11009

Table 2: initial latency and resource utilization on the baseline fixed point of the baseline tiled convolution implementation

3.5 Part 5: Build the HLS IP and deploy on FPGA **20 points**

3.5.1 Code Organization

In this part of the lab, you are going to work on the same code from part4.

3.5.2 Your Task

After choosing your golden fixed point representation of the optimized tiled convolution layer from part 4, now it is time to Accelerate your HLS IP using the **Vitis→Vivado→PYNQ Framework Flow** and report your findings. Follow Tutorial .1 to help you with this task. **Note : You will have to write your own Notebook**

4 What to Submit

- Part 1: **25 points**
  1. tiled_conv.cpp (completed code) 5 points
  2. utils.cpp (completed code) 20 points
  3. screenshot of the simulation of the baseline tiled convolution (MSE ≤ 5)
- Part 2: **20 points**
  1. screenshot of the latency and resource utilization of the baseline tiled convolution
  2. tables, graphs, pie charts, etc . . . whatever you find necessary to document your findings, the goal is to easily visualize your findings. Draw a very solid conclusion, and comment on how the relationship between the tile size and/or the number of kernel groups affect the latency and resource utilization.
- Part 3: 15 points
  1. tiled_conv.cpp (Optimized code) 5 points
  2. utils.cpp (Optimized code) 10 points

3. screenshot of the simulation of the golden tiled convolution
4. screenshot of the latency and resource utilization of the golden tiled convolution

- Part 4: **20 points**

1. tiled_conv.hpp (Fixed point optimized code) 5 points
2. tiled_conv.cpp (completed code)
3. utils.cpp (completed code)
4. tables, graphs, pie charts, etc . . . whatever you find necessary to document your findings, the goal is to easily visualize your findings. Draw a very solid conclusion, and comment on how the relationship between your optimized fixed point golden tile size and/or the number of kernel groups affect the latency and resource utilization.

- Part 5: **20 points**

1. tiled_conv.bit (bitfile code) 5 points
2. tiled_conv.hwh (hardware handoff file) 5 points
3. tiled_conv.ipynb (Jupyter Notebook code) 10 points
4. Compare the Vitis HLS latency estimate (report generated by the tool) of your golden fixed-point optimized convolutional layer vs the actual latency achieved on-board.
5. Compare the speed-up achieved from running the HLS IP on PL vs PS latency for your golden fixed-point optimized convolutional layer.
6. Why is the on-board latency different than the Vitis estimate for unoptimized and optimized IPs?

If you are curious to know, we are using the input image in Figure .6 which is taken from ImageNet1k dataset, and Figure .7 is the output of the first convolution layer in VGG-11 model.



Figure 6: Input image example from the ImageNet1k dataset

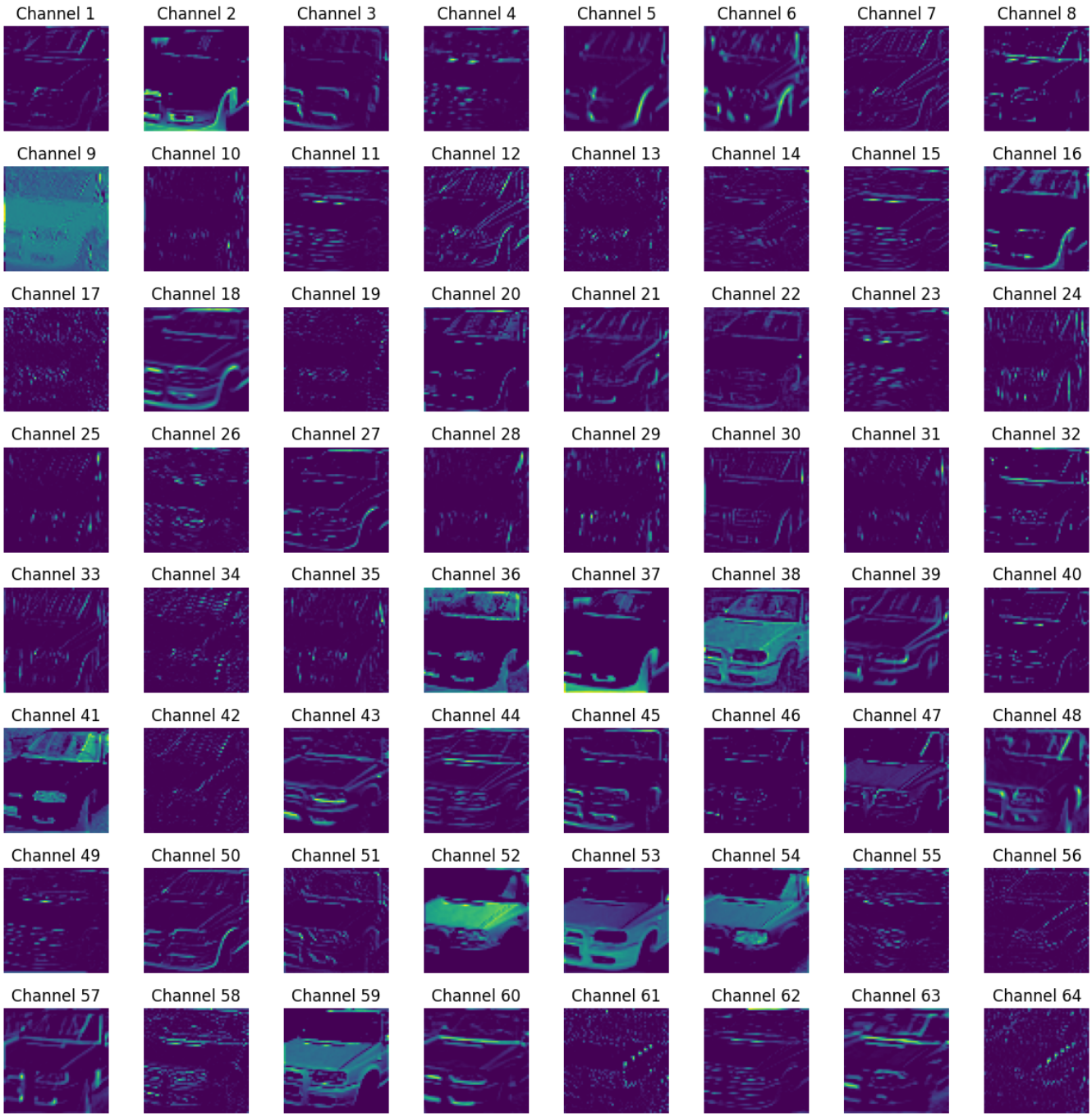


Figure 7: Output of the first convolution layer in VGG-11 model